

Gate 4 — Wiremock API Testing+ Selenide UI

- Participant: R.P.Cipiraj Date: <25.6.26 Branch: <cipiraj-week04-gate04>
- Assessment: Week 4 · Gate 4 —Wiremock API + Selenide UI Testing

1. Objective & user story

In this assessment, there are two jobs. Firstly, that get the consumer's request and expectation as a pact. Generate and Publish the pact to the Pact Broker. Then the Provider verifies the pact and provides the verification results. Based on the Verification results, deployment can be decided.

Secondly, we need to assert the visibility of the 'availability' badge in the cart page, with no manual waits, no thread.sleep.

2. Environment & setup

Layer	Pinned choice
Language / build	Java 21 · Maven · Surefire 3.5.6
HTTP client	RestAssured 6.0.0
Test platform	IntelliJ, Github-Actions
Reporting	Screenshots as Evidence
OS / machine	Windows 11 › · Java 21

Prerequisites & how to run

As we are not using Docker, need to create an account in pactflow.io and get the url and token:

PACT_BROKER_URL

PACT_BROKER_TOKEN

Selenide Dependency

To generate the pacts:

- mvn clean -Dtest=PosOmsConsumerPactTest test

3. Pact

3.1 Consumer - Pacts Generation

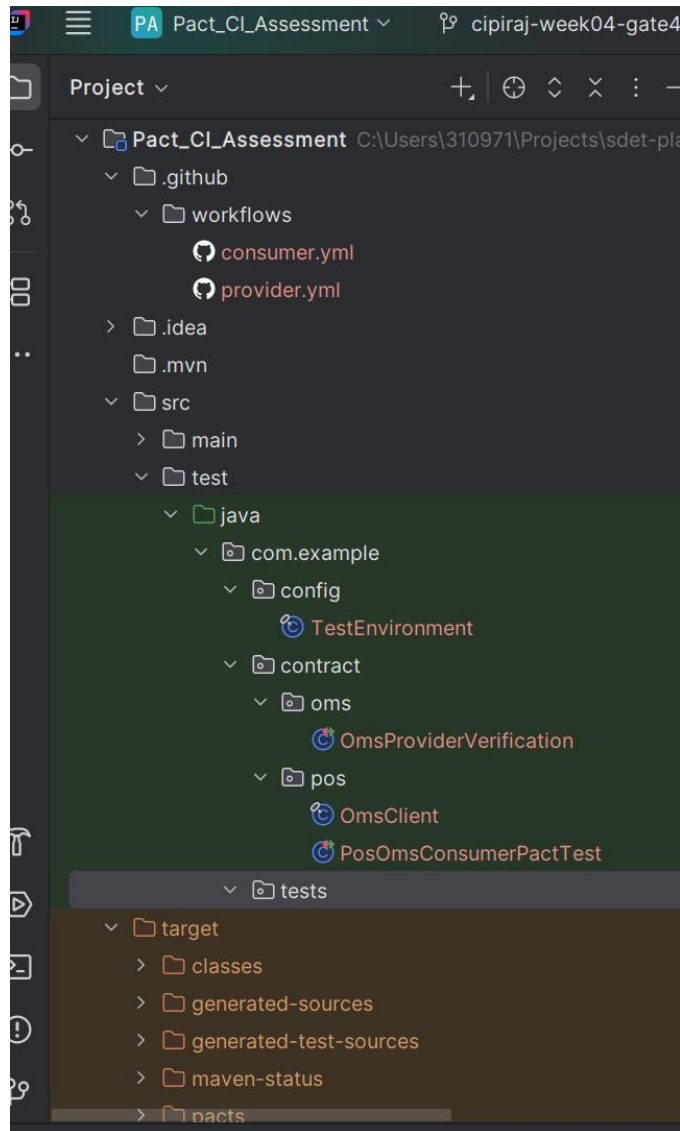
- The Generated Pacts are published to the pact broker
- In the assessment,using OmsClient pacts are getting generated and it is asserted

3.2 Provider- Verifies the pact

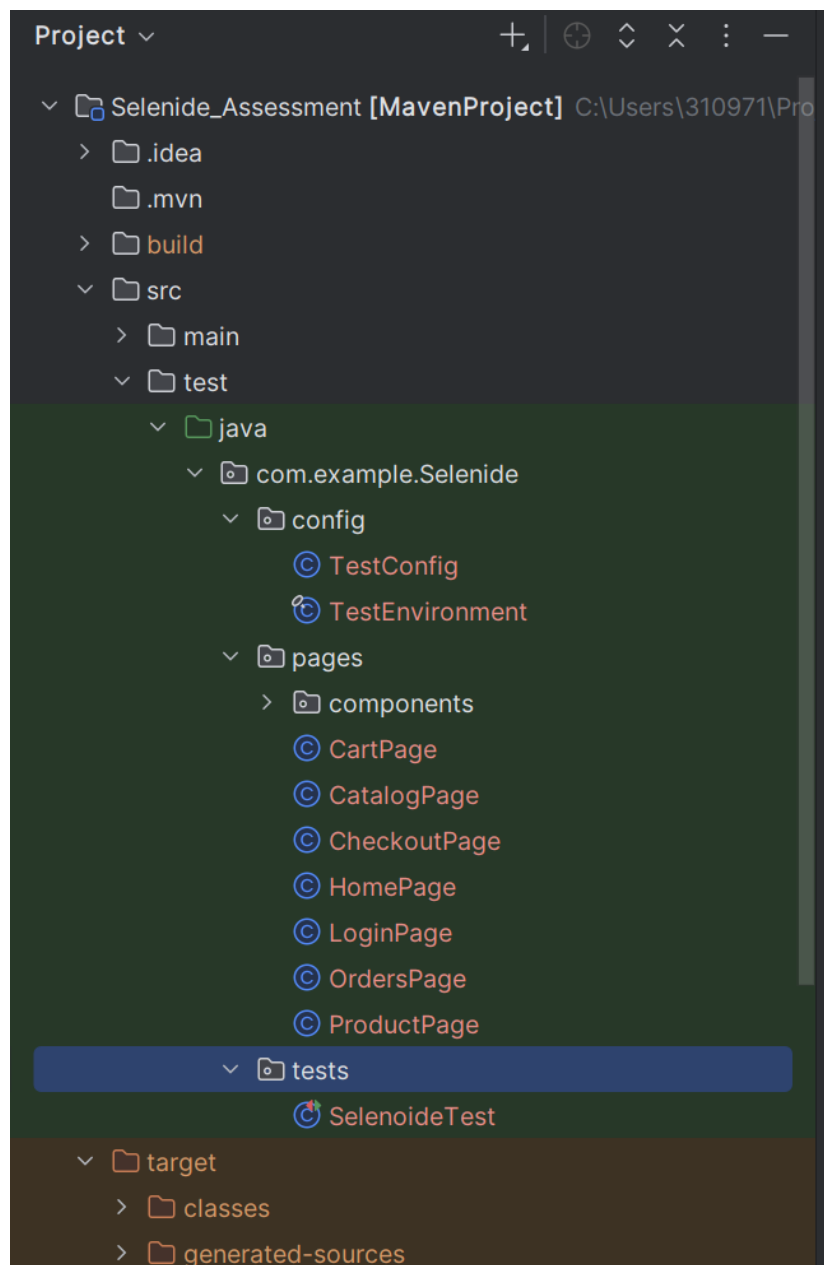
- After the pacts are getting generated,then the pact broker provides it to the provider for the verification

4. Project structure

Wiremock:



Selenide:



5. Test cases covered (3)

Wiremock:

```
@Test
@PactTestFor(pactMethod = "getOrder")
void testGetOrder(MockServer mockServer) {

    OmsClient client =
        new OmsClient(mockServer.getUrl());

    OmsClient.Order order = client.getOrder(id: 123);

    assertEquals(expected: 200, order.statusCode());
    assertEquals(expected: 123, order.orderId());
    assertEquals(expected: "CONFIRMED", order.status());
    assertEquals(expected: 12.0, order.total());
}
```

1.

2.

```
@Test
@PactTestFor(pactMethod = "getInvalidOrder")
void testInvalidGetOrder(MockServer mockServer) {

    OmsClient client =
        new OmsClient(mockServer.getUrl());

    OmsClient.InvalidOrder invalid =
        client.invalidGetOrder(9999);

    assertEquals(expected: 404, invalid.statusCode());
    assertEquals(expected: "Item Not Found", invalid.error());
}
}
```

3. In Selenide

```
static void setup() { TestConfig.apply(); }

@Test
@DisplayName("Module Test of Cart Verification")
void checkAccessibility(){
    HomePage homePage = new HomePage()
        .gotohome();
    LoginPage loginPage = homePage.gotoLogin();
    homePage = loginPage.makeLogin(TestEnvironment.required( name: "EMAIL"),

    CatalogPage catalogPage = homePage.goToCatalogAsLoggedUser()
        .searchProduct(TestEnvironment.required( name: "PRODUCT_NAME"));
```

7. Results

There are 3 Testcase:

In Wiremock asserting 200 and asserting 403

In Seledide asserting the visibility of an Element

All 3 Testcases passed

8. Evidence

Wiremock:

The screenshot displays the GitHub Actions interface for a workflow named 'consumer.yml'. The top navigation bar includes links for Code, Issues, Pull requests, Agents, Actions (selected), Projects, Wiki, Security and quality, Insights, and Settings. The workflow is titled 'Consumer CI' and is marked as 'It is committed #2'. A 'Cancel workflow' button and a 'Latest #2' dropdown are visible. The left sidebar shows the workflow structure with 'Summary' selected, followed by 'All jobs', 'build', 'Run details', 'Usage', and 'Workflow file'. The main area shows a table of jobs with columns for 'Re-run triggered now', 'Status', 'Total duration', and 'Artifacts'. A single job 'CipiRaJRP' is listed with status 'In progress' and a duration of '7s'. Below the table, the 'consumer.yml' file content is shown, indicating it is triggered 'on: push' and contains a 'build' job.

Re-run triggered now	Status	Total duration	Artifacts
CipiRaJRP	In progress	7s	-

Consumer CI

It is committed #2

Re-run all jobs Latest #2

Summary

All jobs

build

Run details

Usage

Workflow file

build succeeded now in 24s

Search logs

- > Set up job 1s
- > Checkout Code 1s
- > Set up Java 7s
- > Install Pact CLI 1s
- > Run Consumer Tests 8s
- > Publish Contracts to PactFlow 2s
- > Can I Deploy (Consumer) 0s
- > Post Set up Java 1s
 - 1 Post job cleanup.
 - 2 Cache hit occurred on the primary key setup-java-linux-x64-maven-3957e54a099ccc21c32ca26568032d6dc66ac9fbe6974b891ab327c8bae9f4d6, not saving cache.

Provider CI

It is committed #2

Re-run all jobs

Summary

All jobs

build

Run details

Usage

Workflow file

build succeeded now in 25s

Search logs

- > Set up job
- > Checkout Code
- > Set up Java
- > Install Pact CLI
- > Run Provider Verification
- > Mark Provider as Deployed
- > Post Set up Java
- > Post Checkout Code
- > Complete job

```

58 [INFO]
59 Error: Tests run: 1, Failures: 0, Errors: 1, Skipped: 0
60 [INFO]
61 [INFO] -----
62 [INFO] BUILD FAILURE
63 [INFO] -----
64 [INFO] Total time: 4.299 s
65 [INFO] Finished at: 2026-06-27T12:40:16Z
66 [INFO] -----
67 Error: Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.5.6:test (default-test)
MavenProject:
68 Error:
69 Error: See /home/runner/work/PactProjectwithNegativeMapping/PactProjectwithNegativeMapping/target/suref
the individual test results.
70 Error: See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
71 Error: -> [Help 1]
72 Error:
73 Error: To see the full stack trace of the errors, re-run Maven with the -e switch.
74 Error: Re-run Maven using the -X switch to enable full debug logging.
75 Error:
76 Error: For more information about the errors and possible solutions, please read the following articles
77 Error: [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
78 Error: Process completed with exit code 1.

```

Broker API due to intentional failure done in Provider Side

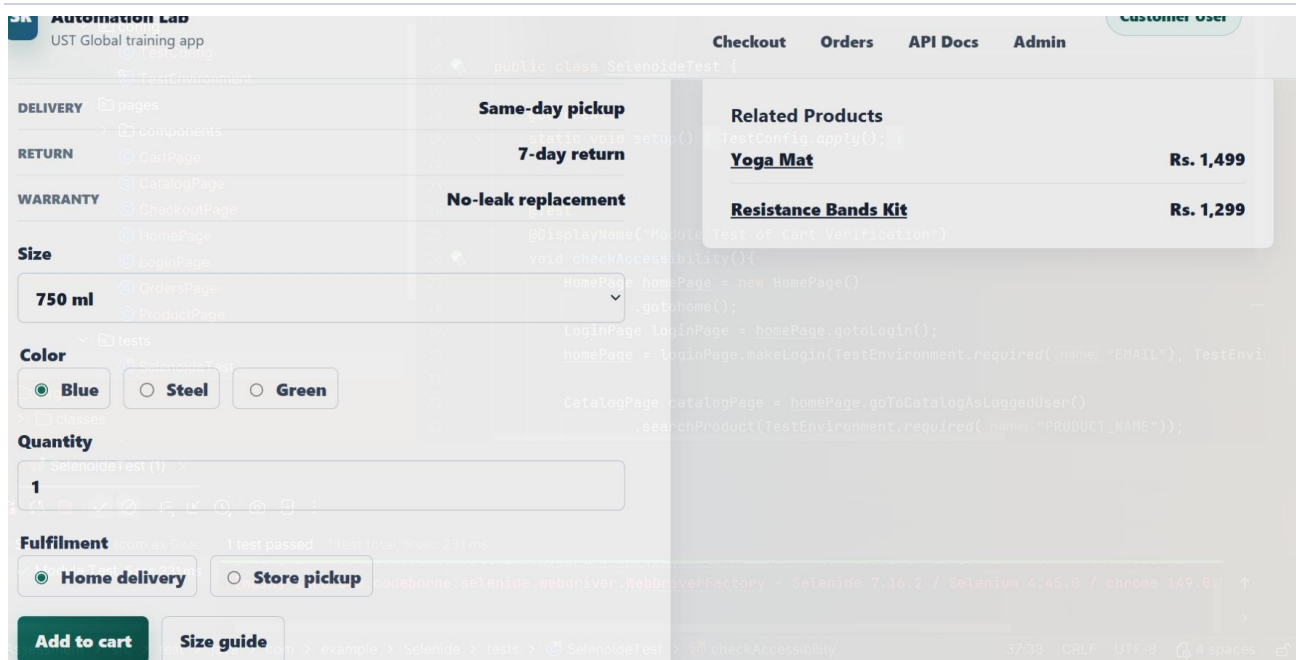
```

    """)));
}

@State("Order 9999 does not exists") no usages @Cipiraj Rangaraj Parimaladevi(UST
void isOrderNotExists(){
    wireMock.stubFor(get(urlEqualTo(testUrl: "/orders/9999")))
        .willReturn(aResponse()
            .withStatus(404)
            .withHeader(key: "Content-Type", ...values: "application/json")
            .withBody("""
                {"status":"CONFIRMED"}
                """)));
}
}

```

Selenium



9. Github Repository

<https://github.com/CipiRaJRP/PactProjectwithNegativeMapping>

10. Conclusion — what this proves

A Selenide Verification and Wiremock consumer and Provider verification is done by CI/CD